

Prueba Técnica

Desarrollador Backend

Java + Spring Boot (REST)

Stack Tecnológico Requerido

Capa	Tecnología
Lenguaje	Java 17+
Framework	Spring Boot 3.x
API	Spring Web MVC (REST / JSON)
ORM	Spring Data JPA + Hibernate
Base de Datos	MySQL o PostgreSQL
Tokens / Sesión	UUID + token numérico de 6 dígitos
Correo	Spring Boot Starter Mail (JavaMailSender)
Build Tool	Maven o Gradle
Contenedores	Docker + Docker Compose (recomendado)
Docs API	Springdoc OpenAPI / Swagger UI
Colección	Postman Collection exportada

Descripción del Problema

Se debe construir una API REST con Spring Boot que simule una billetera virtual. La API es el único punto de acceso tanto para el frontend como para cualquier cliente externo.

El sistema debe soportar las siguientes operaciones:

1. Registrar un cliente.
2. Cargar dinero a la billetera.
3. Iniciar una compra (genera token de confirmación enviado por email).
4. Confirmar el pago con el token recibido.
5. Consultar el saldo de la billetera.

Arquitectura del Proyecto

Se valorará una estructura limpia, modular y con separación clara de responsabilidades:

```
wallet-api/
├── src/main/java/com/wallet/
│   ├── controller/      # @RestController – recibe peticiones HTTP
│   ├── service/         # Lógica de negocio (@Service)
│   ├── repository/      # Acceso a datos (@Repository / JpaRepository)
│   ├── entity/          # Entidades JPA (@Entity)
│   ├── dto/             # Objetos de transferencia (Request / Response)
│   ├── exception/       # Excepciones personalizadas + @ControllerAdvice
│   └── config/           # Configuración (Mail, Seguridad, OpenAPI)
├── src/main/resources/
│   └── application.yml
├── src/test/
├── Dockerfile
├── docker-compose.yml
└── README.md
```

Estructura Estándar de Respuesta

Todas las respuestas (éxito y error) deben seguir la misma estructura JSON. Se valorará el uso de un wrapper genérico con un tipo genérico T:

```
{
  "success": true,
  "cod_error": "00",
  "message_error": "Operación exitosa",
  "data": { }
}
```

Códigos de error sugeridos:

Código	Significado
00	Operación exitosa
01	Error de validación / campos requeridos
02	Cliente no encontrado
03	Saldo insuficiente
04	Token inválido o expirado
99	Error interno del servidor

Funcionalidades Requeridas

1. Registrar Cliente — POST /api/clientes/registrar

Registra un nuevo cliente en el sistema. Todos los campos son obligatorios.

Request Body:

```
{
  "documento": "123456789",
  "nombres": "Juan Pérez",
  "email": "juan@email.com",
  "celular": "3001234567"
}
```

Validaciones:

- Todos los campos son requeridos (@NotBlank).
- El email debe tener formato válido (@Email).
- El documento debe ser único en la base de datos.

2. Recargar Billetera — POST /api/billetera/recargar

Permite agregar saldo a la billetera de un cliente existente.

Request Body:

```
{
  "documento": "123456789",
  "celular": "3001234567",
  "valor": 50000.00
}
```

Validaciones:

- El par documento + celular debe coincidir con un cliente registrado.
- El valor debe ser mayor a 0 (@Positive).

3. Iniciar Pago — POST /api/billetera/pagar

Inicia una transacción de compra. No descuenta el saldo inmediatamente; espera confirmación.

Request Body:

```
{
  "documento": "123456789",
  "celular": "3001234567",
  "valor": 15000.00
}
```

Flujo esperado:

6. Verificar que el cliente exista (documento + celular).
7. Verificar que el saldo sea suficiente para cubrir el valor.
8. Generar un token numérico de 6 dígitos aleatorio.
9. Generar un sessionId único (UUID).
10. Persistir la transacción con estado PENDIENTE y tiempo de expiración (ej. 15 minutos).
11. Enviar el token al email del cliente con JavaMailSender.
12. Retornar el sessionId en la respuesta.

Snippet sugerido:

```
String token      = String.format("%06d", new Random().nextInt(999999));
String sessionId = UUID.randomUUID().toString();
```

Response data:

```
{
  "sessionId": "a1b2c3d4-...",
  "mensaje": "Se ha enviado un código de confirmación a su email"
}
```

4. Confirmar Pago — POST /api/billetera/confirmar-pago

Valida el token recibido por email y completa el descuento del saldo.

Request Body:

```
{
  "sessionId": "a1b2c3d4-...",
  "token": "482931"
}
```

Flujo esperado:

13. Buscar la transacción pendiente por sessionId.
14. Verificar que el token coincida.
15. Verificar que la transacción no haya expirado.
16. Descontar el valor del saldo de la billetera.
17. Marcar la transacción como COMPLETADA.
18. Retornar mensaje de éxito o el error correspondiente.

5. Consultar Saldo — GET /api/billetera/saldo

Retorna el saldo actual de la billetera.

Body Request:

```
GET /api/billetera/saldo
```

```
{
  "documento": 123456789,
  "celular": "3001234567"
}
```

Validaciones:

- Ambos parámetros son obligatorios.
- El par documento + celular debe coincidir con un cliente registrado.

Response data:

```
{
  "saldo": 35000.00,
  "nombres": "Juan Pérez"
}
```

Entidades JPA

Cliente

```
@Entity
@Table(name = "clientes")
public class Cliente {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true, nullable = false)
    private String documento;
    @Column(nullable = false)
    private String nombres;
    @Column(nullable = false)
    private String email;
    @Column(nullable = false)
    private String celular;
}
```

Billetera

```
@Entity
@Table(name = "billeteras")
```

```

public class Billetera {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "cliente_id")
    private Cliente cliente;
    @Column(precision = 15, scale = 2, nullable = false)
    private BigDecimal saldo = BigDecimal.ZERO;
}

```

TransaccionPendiente

```

@Entity
@Table(name = "transacciones_pendientes")
public class TransaccionPendiente {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, unique = true)
    private String sessionId;
    @Column(nullable = false)
    private String token;
    @Column(nullable = false, precision = 15, scale = 2)
    private BigDecimal valor;
    @ManyToOne(fetch = FetchType.LAZY)
    private Cliente cliente;
    private LocalDateTime expiracion;
    @Enumerated(EnumType.STRING)
    private EstadoTransaccion estado; // PENDIENTE, COMPLETADA, EXPIRADA
}

```

Resumen de Endpoints REST

Método	Ruta	Descripción
POST	/api/clientes/registrar	Registrar nuevo cliente
POST	/api/billetera/recargar	Recargar saldo de la billetera
POST	/api/billetera/pagar	Iniciar pago (genera token y sessionId)
POST	/api/billetera/confirmar-pago	Confirmar pago con sessionId + token
GET	/api/billetera/saldo	Consultar saldo (params: documento, celular)

Buenas Prácticas a Aplicar

- Arquitectura en capas: Controller → Service → Repository. No poner lógica de negocio en el Controller.
- DTOs para request y response; nunca exponer las entidades JPA directamente al cliente.
- Manejo global de excepciones con `@ControllerAdvice` y `@ExceptionHandler`.
- Validaciones declarativas con Bean Validation: `@NotBlank`, `@Email`, `@Positive`, `@NotNull`.
- Inyección de dependencias `@RequiredArgsConstructor`.
- Configuración sensible (credenciales, puerto SMTP) en variables de entorno, no hardcodeada.
- Logs estructurados con `@Log4j2` en cada operación relevante.

Dependencias Maven Clave (pom.xml)

```
<dependencies>
  <!-- Web / REST -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- JPA + MySQL -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
  </dependency>

  <!-- Validación -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>

  <!-- Mail -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
  </dependency>

  <!-- Swagger / OpenAPI -->
  <dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  </dependency>
</dependencies>
```

```

    <version>2.3.0</version>
  </dependency>

  <!-- Lombok (opcional) -->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>

  <!-- Test -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

```

Entregables

19. Repositorio Git con commits por funcionalidad completada.
20. README.md completo con instrucciones de instalación y ejecución local.
21. docker-compose.yml para levantar la API y la base de datos.
22. Colección de Postman exportada con documentación de cada endpoint.
23. Swagger UI disponible en /swagger-ui.html.
24. Script SQL de inicialización de BD.

Tiempo y Entrega

Tiempo máximo	
Control de versiones	Git — un commit por funcionalidad completada
Montaje	El proyecto debe poder levantarse fácilmente; si no se puede evaluar, no se califica
Video	Máximo 15 minutos mostrando funcionalidades y componentes

Criterios de Evaluación

Criterio	Peso
----------	------

Correcto funcionamiento de los 5 endpoints REST	Alto
Arquitectura en capas (Controller / Service / Repository)	Alto
Validaciones con Bean Validation y manejo global de errores	Alto
Flujo completo de pago con token + email (JavaMailSender)	Alto
Buenas prácticas y código limpio	Alto
Frontend funcional en React JS o Vue JS	Medio
Docker Compose funcional	Medio
Pruebas unitarias (JUnit 5 + Mockito)	Medio
Swagger UI + Postman Collection documentada	Medio
Claridad del README y facilidad de montaje	Bajo

¡Mucho éxito! Se valorará la calidad del código sobre la velocidad de entrega.